

Plano de teste - API de autenticação

1. Objetivo

Garantir que a rota `/produto/<id>` funcione corretamente ao retornar dados de um produto específico e trate os erros de forma adequada.

2. Escopo

Endpoint: `/create`, `/read`, `/update`, `/delete`

`/create`: Cria um novo registro no sistema.

Exemplo:

- Quando você cadastra um novo produto, cliente ou usuário.
- Na API: POST `/produto` cria um novo produto.

`/read`: Lê ou consulta dados já existentes.

Exemplo:

- Quando você acessa a lista de produtos ou vê os detalhes de um produto.
- Na API: GET `/produto` ou GET `/produto/<id>`

`/update`: Atualiza dados de um registro que já existe.

Exemplo:

- Quando você muda o nome ou o preço de um produto.
- Na API: PUT `/produto/<id>`

`/delete`: Exclui um registro do sistema.

Exemplo:

- Quando você remove um produto do estoque.
- Na API: DELETE `/produto/<id>`

3. Equipe

QA: Elysa

Backend: Mester

4. Métricas

100% de cobertura para variações de ID (válido, inválido, inexistente)

< 5% de falhas nos testes automatizados

5. Casos de Teste

Caso	Descrição	Entrada	Resultado Esperado
CT01	ID válido existente	GET /produto/1	Status 200 + JSON com os dados do produto
CT02	ID numérico inexistente	GET /produto/999	Status 404 + mensagem "Produto não encontrado"
CT03	ID não numérico	GET /produto/abc	Status 400 ou 422 + erro de validação
CT04	ID negativo	GET /produto/-5	Status 400 ou 422 + erro indicando ID inválido
CT05	Rota incompleta sem ID	GET /produto/	Status 404 ou 405 (rota não encontrada ou método inválido)

Requisito Funcional: Criar Produto

- Endpoint: **POST /produto**
- Descrição: O sistema deve permitir o cadastro de novos produtos com nome, preço e quantidade.
- Critérios de Aceitação:
 - Retornar **201 Created** se sucesso

- Retornar **400 Bad Request** para dados inválidos ou campos vazios
- Retornar **409 Conflict** se o produto já existir (caso aplicável)

Requisito Funcional: Consultar Produto

- Endpoint: **GET /produto/<id>**
- Descrição: O sistema deve retornar os detalhes de um produto específico.
- Critérios de Aceitação:
 - **200 OK** se sucesso
 - **400 Bad Request** se ID for inválido
 - **404 Not Found** se produto não existir

Requisito Funcional: Atualizar Produto

- Endpoint: **PUT /produto/<id>**
- Descrição: O sistema deve permitir editar um produto já cadastrado.
- Critérios de Aceitação:
 - **200 OK** se sucesso
 - **400 Bad Request** se os dados forem inválidos
 - **404 Not Found** se o produto não for encontrado

Requisito Funcional: Deletar Produto

- Endpoint: **DELETE /produto/<id>**
- Descrição: O sistema deve permitir excluir um produto pelo ID.
- Critérios de Aceitação:
 - **200 OK** se excluído com sucesso
 - **404 Not Found** se o ID não existir

CRUD – Possíveis Respostas HTTP

Create:POST

- ✓ 201 Created: produto criado com sucesso
- ⚠ 400 Bad Request: dados inválidos ou faltando
- ⚠ 409 Conflict: item já existe (caso duplicado)

Read:GET

- ✓ 200 OK: dados retornados com sucesso
- ⚠ 400 Bad Request: ID inválido
- ⚠ 404 Not Found: produto não encontrado

Update:PUT

- ✓ 200 OK: produto atualizado
- ⚠ 400 Bad Request: dados inválidos
- ⚠ 404 Not Found: produto não existe

Delete:DELETE

- ✓ 200 OK ou 204 No Content: produto removido
- ⚠ 404 Not Found: produto não encontrado